

AEWIS-Saucer — Web Dashboard PRD

Version 3.0 · Web Only · Neo4j · No Docker

Date: May 2026

Stack: Node.js + Express · Neo4j (native install) · MQTT · Vite + React
· Chart.js · Socket.io

Database: Neo4j 5.x (no Docker — systemd service)

Deployment: Ubuntu 22.04 VPS · Nginx · PM2 · Certbot SSL

1. What This PRD Covers

This is the **web dashboard only** PRD. It covers: - The complete React web frontend - Node.js + Express backend API - Neo4j graph database schema (no Docker) - MQTT → backend → Neo4j data pipeline - Real-time via Socket.io - Every screen, every feature, every API endpoint - Full test cases

It does NOT cover: mobile app, Docker, Kubernetes, or enterprise multi-tenant.

2. Why Neo4j (No Docker)

Neo4j is a graph database. It models relationships between entities naturally:

```
(User)-[:OWNS]->(Device)-[:HAS_READING]->(Reading)
(Device)-[:LOCATED_AT]->(Location)
(Reading)-[:TRIGGERED]->(Alert)
(User)-[:CONFIGURED]->(AlertRule)
```

This makes queries like “show me all alerts triggered by devices in my lab” or “find all readings above threshold for this device in the last hour” extremely fast and expressive.

Neo4j without Docker — install directly on Ubuntu:

```
# Add Neo4j repo
wget -O - https://debian.neo4j.com/neotechnology.gpg.key | sudo apt-key add -
echo 'deb https://debian.neo4j.com stable 5' | sudo tee /etc/apt/sources.list.d/neo4j.list
sudo apt update
sudo apt install neo4j
```

```
# Start as systemd service (no Docker needed)
sudo systemctl enable neo4j
sudo systemctl start neo4j
```

```
# Neo4j runs at bolt://localhost:7687
# Web console at http://localhost:7474
```

npm package to connect:

```
npm install neo4j-driver
```

3. Neo4j Graph Data Model

Nodes

Node Label	Properties	Purpose
User	id, email, name, passwordHash, role, createdAt	Account holder
Device	id, name, location, type, firmwareVersion, lastSeen, status	AQM hardware unit
Reading	id, ts, pm25, pm10, pm1, co, no2, co2, o3, voc, temp, rh, aqi, primaryPollutant	Single sensor snapshot
Alert	id, ts, pollutant, value, threshold, level, acknowledged	Threshold breach event
AlertRule	id, pollutant, warnThreshold, critThreshold, cooldownMin	User-defined thresholds
Location	id, name, lat, lon, address	Geographic place

Relationships

Relationship	From → To	Properties
OWNS	User → Device	claimedAt
HAS_READING	Device → Reading	—
TRIGGERED	Reading → Alert	—
CONFIGURED	User → AlertRule	—
APPLIES_TO	AlertRule → Device	—
LOCATED_AT	Device → Location	installedAt

Key Cypher Queries

```
// Get last 100 readings for a device
MATCH (d:Device {id: $deviceId})-[:HAS_READING]->(r:Reading)
```

```

RETURN r ORDER BY r.ts DESC LIMIT 100

// Get all devices with current reading for a user
MATCH (u:User {id: $userId})-[:OWNS]->(d:Device)
OPTIONAL MATCH (d)-[:HAS_READING]->(r:Reading)
WITH d, r ORDER BY r.ts DESC
WITH d, COLLECT(r)[0] AS latest
RETURN d, latest

// Get alerts for last 7 days
MATCH (d:Device {id: $deviceId})-[:HAS_READING]->(r:Reading)-[:TRIGGERED]->(a:Alert)
WHERE a.ts > datetime() - duration('P7D')
RETURN a ORDER BY a.ts DESC

// Average AQI by hour for last 24h
MATCH (d:Device {id: $deviceId})-[:HAS_READING]->(r:Reading)
WHERE r.ts > datetime() - duration('P1D')
RETURN r.ts.hour AS hour, AVG(r.aqi) AS avgAqi ORDER BY hour

```

4. Full Tech Stack

Layer	Technology	Version	Purpose
Frontend	React 18 + Vite	18.x / 5.x	Web app
Routing	React Router v6	6.x	Client-side routing
Charts	Chart.js 4	4.4	All charts and sparklines
Maps	Leaflet.js	1.9	Device location map
Real-time	Socket.io-client	4.x	Live data stream
Styling	Tailwind CSS v3	3.x	Utility CSS
State	Zustand	4.x	Global state
HTTP client	Axios	1.x	REST API calls
Backend	Node.js + Express	20.x / 4.x	REST API + WS server
Real-time	Socket.io	4.x	Push to frontend
MQTT client	mqtt.js	5.x	Receive from ESP32
Database	Neo4j 5 + neo4j-driver	5.x	Graph database
Auth	jsonwebtoken + bcrypt	—	JWT auth
Email	Nodemailer + Gmail/SMTP	—	Alert emails
Validation	Zod	3.x	Input validation
Process mgr	PM2	—	Keep Node.js alive
Web server	Nginx	—	Reverse proxy + SSL
SSL	Certbot (Let's Encrypt)	—	HTTPS
Deployment	Ubuntu 22.04 VPS	—	No Docker

5. Project File Structure (Single Project)

```
aewis-web/  
  frontend/                                ← React app (Vite)  
    src/  
      pages/  
        Dashboard.jsx                     ← Main live dashboard  
        GasDetail.jsx                     ← Single gas drill-down  
        History.jsx                       ← Historical data + charts  
        Alerts.jsx                        ← Alert management  
        Devices.jsx                       ← Device management  
        Settings.jsx                     ← Account + device settings  
        Login.jsx                         ← Auth pages  
        Register.jsx  
      components/  
        GasGauge.jsx                     ← Arc gauge component  
        AQIRing.jsx                     ← Large AQI ring  
        TrendChart.jsx                   ← Multi-line Chart.js  
        SparkLine.jsx                   ← Mini sparkline per gas  
        AlertBanner.jsx                 ← Top alert strip  
        DeviceStatus.jsx                 ← Online/offline badge  
        ExportModal.jsx                 ← Export dialog  
      hooks/  
        useSocket.js                     ← Socket.io live data  
        useDevice.js                     ← Device data + polling  
        useAlerts.js                     ← Alert state  
      store/  
        index.js                         ← Zustand global store  
      utils/  
        aqi.js                           ← AQI calculation  
        thresholds.js                     ← WHO/EPA constants  
        formatters.js                     ← Number/date formatting  
      api/  
        client.js                         ← Axios instance + interceptors  
    public/  
      index.html  
      vite.config.js  
  
  backend/                                ← Node.js + Express API  
    src/  
      server.js                           ← Express app entry  
      mqtt/  
        subscriber.js                     ← MQTT → Neo4j pipeline  
      routes/  
        auth.js                           ← /api/auth/*  
        devices.js                         ← /api/devices/*
```

readings.js	← /api/readings/*
alerts.js	← /api/alerts/*
export.js	← /api/export/*
services/	
neo4j.js	← Neo4j driver + helpers
alertEngine.js	← Threshold checking
emailService.js	← Nodemailer
aqiCalc.js	← AQI computation
middleware/	
auth.js	← JWT verify middleware
rateLimit.js	← Rate limiting
validate.js	← Zod validation
socket/	
emitter.js	← Socket.io emit helpers
.env	
ecosystem.config.js	← PM2 config
nginx.conf	← Nginx config
package.json	← Root scripts

6. Dashboard Screens — Every Feature

6.1 Main Dashboard (Home Screen)

Layout — what user sees on load:

```

AEWIS  LIVE  [device selector ]  12:34:56

      Primary Pollutant  Status
AQI Ring  Readings Today  24H Avg AQI
(big, color)

      Alert Banner (green/orange/red)

REAL-TIME GAS MONITOR - ALL 6 CHANNELS

PM2.5    CO    NO    CO    O    VOC
gauge    gauge gauge gauge gauge
spark    spark spark spark spark

Temp: 28.3°C           Humidity: 55.2%

TREND - ALL GASES NORMALISED  [1H] [6H] [24H]
  legend

```

[Chart.js multi-line - all 6 gases on one chart]

Each gas card shows: - Gas name + sensor chip name - Colored arc gauge (0% → max) - Current value (large, monospace) - Unit - Status badge: GOOD / MOD / HIGH / CRIT - 60-point sparkline (last 2 minutes) - Click card → goes to Gas Detail page

Main trend chart features: - All 6 gases plotted as % of their warning threshold (normalized) - 100% line = warning threshold - Time range: 1H / 6H / 24H - Hover tooltip: all 6 gas values at that timestamp - Click legend → highlight/hide that gas - Chart.js 4, line type, smooth curves, filled area under active gas

Device selector (top bar): - Dropdown shows all user's devices with on-line/offline status - Selecting different device → refreshes all data without page reload

6.2 Gas Detail Page

URL: `/gas/:gasKey` (e.g. `/gas/pm25`)

Features: - Full-width large line chart for this gas only - Y-axis: actual value + WHO/EPA threshold lines drawn - Time range: 1H / 6H / 24H / 7D / 30D - Stats row: Min / Max / Avg / Current for selected period - Health impact section: description of this pollutant's effects - WHO guideline comparison: "Your 24H avg vs WHO guideline" - Alert history for this pollutant: list of past breaches - Export button: CSV for this gas only

6.3 History Page

URL: `/history`

Features: - Calendar heatmap: each day colored by average AQI (like GitHub contributions) - Click any day → loads that day's full trend - Date range picker: custom start/end date - Comparison mode: select 2 time periods → overlay charts - Multi-gas chart for selected period - Summary stats table: pollutant averages per selected period - Export section (see Section 7)

6.4 Alerts Page

URL: `/alerts`

Features: - Two tabs: Active Alerts / Alert History - Active alerts: any current threshold breach, with time-since - History table: columns = Timestamp, Device, Pollutant, Value, Threshold, Level, Status - Filter: by pollutant, date range, severity (Info/Warning/Critical) - Search bar - Acknowledge button per alert - Bulk acknowledge all - Alert rules section (bottom): configure thresholds per pollutant

Alert rule editor: - Slider per pollutant with WHO/EPA default - Warning threshold + Critical threshold - Cooldown period (minutes between repeated alerts) - Notification channel checkboxes: Email / In-app / Webhook - Save → POST /api/alerts/rules

6.5 Devices Page

URL: /devices

Features: - Cards grid: one card per device - Card shows: device name, location, AQI badge, online/offline, last seen, firmware version - Add Device button → opens provisioning guide - Click device → opens device settings panel - Device settings: rename, change location, reading interval, sensor toggles, Wi-Fi change, factory reset, delete

6.6 Settings Page

URL: /settings

Sections: - Account: name, email, change password, delete account - Notifications: email preferences, alert quiet hours, weekly summary toggle - Data: data retention display, export all data, delete all data - API Keys: generate/revoke keys (for advanced users) - Appearance: Dark / Light theme toggle, temperature unit (°C / °F) - About: firmware versions, contact support

7. Data Export

CSV Export

- URL: GET /api/export/csv?deviceId=&from=&to=&interval=raw|1min|5min|1hour
- Columns: timestamp, pm1, pm25, pm10, co, no2, co2, o3, voc, temp, rh, aqi, aqi_category
- Large exports (>10,000 rows): async job → download link emailed

PDF Report

- URL: POST /api/export/pdf with { deviceId, from, to }
- Generated server-side using PDFKit (npm install pdfkit)
- Contents: summary stats, per-gas charts (Chart.js rendered server-side), alert log, WHO comparison table
- Delivered as download in browser

8. Full API Reference

Auth Routes

Method	Endpoint	Body	Response
POST	/api/auth/register	{email, password, name}	{user, token, refreshToken}
POST	/api/auth/login	{email, password}	{user, token, refreshToken}
POST	/api/auth/refresh	{refreshToken}	{token}
POST	/api/auth/logout	—	200 OK
POST	/api/auth/forgot-password	{email}	200 OK (email sent)
POST	/api/auth/reset-password	{token, newPassword}	200 OK

Device Routes

Method	Endpoint	Description
GET	/api/devices	List all user's devices with latest reading
POST	/api/devices/claim	Claim device by device_id
GET	/api/devices/:id	Device details + current reading
PATCH	/api/devices/:id	Update device name/location/settings
DELETE	/api/devices/:id	Remove device from account
GET	/api/devices/:id/status	Online/offline + connection info
POST	/api/devices/:id/wifi	Trigger Wi-Fi re-provisioning
POST	/api/devices/:id/ota	Trigger OTA firmware update
POST	/api/devices/:id/reset	Factory reset device

Reading Routes

Method	Endpoint	Query Params	Description
GET	/api/readings/latest	deviceId	Latest single reading
GET	/api/readings	deviceId, from, to, interval, limit	Historical readings
GET	/api/readings/stats	deviceId, from, to	Min/Max/Avg per pollutant
GET	/api/readings/heatmap	deviceId, year	Daily AQI averages for calendar
GET	/api/readings/hourly	deviceId, date	Hourly averages for one day

Alert Routes

Method	Endpoint	Description
GET	/api/alerts	List alert history (filterable)
GET	/api/alerts/active	Currently active (unresolved) alerts
PATCH	/api/alerts/:id/ack	Acknowledge an alert
POST	/api/alerts/ack-all	Acknowledge all active alerts
GET	/api/alerts/rules	Get user's configured thresholds
PUT	/api/alerts/rules	Update all thresholds at once

Export Routes

Method	Endpoint	Description
GET	/api/export/csv	Synchronous CSV (10,000 rows)
POST	/api/export/pdf	Generate PDF report (async)
GET	/api/export/status/:jobId	Check export job status
GET	/api/export/download/:jobId	Download completed export

9. Real-Time Architecture

ESP32-S3

publishes JSON → MQTT Broker (Mosquitto)

mqtt.js subscriber (Node.js)

Neo4j write
(persist) Socket.io emit
 (to all connected
 browser clients
 watching this device)

React frontend
useSocket() hook
→ updates Zustand store
→ all components re-render

Socket.io events:

Event	Direction	Payload
sensor:update	Server → Client	Full reading object
alert:new	Server → Client	Alert object
alert:resolved	Server → Client	Alert id

Event	Direction	Payload
device:status	Server → Client	{deviceId, status}
subscribe	Client → Server	{deviceId} — join device room
unsubscribe	Client → Server	{deviceId}

10. Neo4j Setup on Ubuntu (No Docker)

Step 1: Install Neo4j 5

```
wget -O - https://debian.neo4j.com/neotechnology.gpg.key | sudo apt-key add -
echo 'deb https://debian.neo4j.com stable 5' | sudo tee /etc/apt/sources.list.d/neo4j.list
sudo apt update && sudo apt install neo4j -y
```

Step 2: Set password

```
sudo neo4j-admin dbms set-initial-password yourpassword
```

Step 3: Configure

```
sudo nano /etc/neo4j/neo4j.conf
# Uncomment: server.bolt.listen_address=:7687
# Uncomment: server.http.listen_address=:7474
# Set: server.default_listen_address=0.0.0.0 (only if remote access needed)
```

Step 4: Start and enable

```
sudo systemctl enable neo4j
sudo systemctl start neo4j
sudo systemctl status neo4j
```

Step 5: Verify

```
curl http://localhost:7474 # should return Neo4j browser HTML
```

Step 6: Connect from Node.js

```
npm install neo4j-driver
```

backend/src/services/neo4j.js:

```
const neo4j = require('neo4j-driver');

const driver = neo4j.driver(
  'bolt://localhost:7687',
  neo4j.auth.basic('neo4j', process.env.NEO4J_PASSWORD)
);

async function runQuery(cypher, params = {}) {
  const session = driver.session();
  try {
```

```

    const result = await session.run(cypher, params);
    return result.records;
  } finally {
    await session.close();
  }
}

module.exports = { driver, runQuery };

```

11. PM2 Process Management (No Docker)

ecosystem.config.js (at project root):

```

module.exports = {
  apps: [
    {
      name: 'aewis-api',
      script: './backend/src/server.js',
      env: { NODE_ENV: 'production', PORT: 3000 },
      instances: 2,
      exec_mode: 'cluster',
      max_memory_restart: '500M',
      error_file: './logs/api-error.log',
      out_file: './logs/api-out.log',
    }
  ]
};

npm install -g pm2
pm2 start ecosystem.config.js
pm2 startup    # auto-start on server reboot
pm2 save

```

12. Nginx Configuration (No Docker)

```

# /etc/nginx/sites-available/aewis

server {
    listen 80;
    server_name app.aewis.in;
    return 301 https://$host$request_uri;
}

server {

```

```

listen 443 ssl;
server_name app.aewis.in;

ssl_certificate /etc/letsencrypt/live/app.aewis.in/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/app.aewis.in/privkey.pem;

# Serve React frontend (built with vite build)
root /var/www/aewis/frontend/dist;
index index.html;
try_files $uri $uri/ /index.html;    # SPA routing

# Proxy API calls to Node.js
location /api/ {
    proxy_pass http://localhost:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}

# Proxy Socket.io WebSocket
location /socket.io/ {
    proxy_pass http://localhost:3000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
}
}

sudo certbot --nginx -d app.aewis.in
sudo nginx -t && sudo systemctl reload nginx

```

13. Complete Test Cases — Web Dashboard

Frontend Unit Tests

Component	Test	Expected
GasGauge	PM2.5 = 18.7, max = 100	Arc fills 18.7% of range
GasGauge	Value above critical threshold	Badge shows “CRIT”, red color
AQIRing	AQI = 72	Arc 18% filled, yellow color
AQIRing	AQI = 0	Arc empty, green label “Good”

Component	Test	Expected
TrendChart	Empty history array	Chart renders with empty dataset, no crash
TrendChart	60 data points	All 60 plotted, no overflow
AlertBanner	No active alerts	Shows green “All safe” message
AlertBanner	CO critical	Shows red CO alert text
SparkLine	2 data points	Renders minimal line, no crash
DeviceStatus	status = “offline”	Shows grey dot + “Offline”
formatters.js	fmt(18.7, 1)	“18.7”
formatters.js	fmt(0.04123, 3)	“0.041”
aqi.js	calcAQI({pm25: 18.7})	Returns ~68
aqi.js	calcAQI({pm25: 250})	Returns 200
thresholds.js	getStatus('pm25', 14)	{level: 'Good', color: '#22C55E'}
thresholds.js	getStatus('pm25', 40)	{level: 'Moderate', ...}

API Integration Tests

Endpoint	Test	Expected
POST /api/auth/login	Valid credentials	200, JWT + refreshToken
POST /api/auth/login	Wrong password	401, error message
POST /api/auth/login	Missing email field	400, validation error
GET /api/devices	Valid JWT	200, array of devices
GET /api/devices	No JWT	401 Unauthorized
GET /api/devices	Expired JWT	401, “token expired”
GET /api/readings	deviceId + from + to	200, readings array
GET /api/readings	deviceId not owned by user	403 Forbidden
GET /api/readings	Invalid date format	400, validation error
GET /api/readings/stats	7 days range	200, min/max/avg per gas
GET /api/alerts/active	No active alerts	200, empty array
PUT /api/alerts/rules	Valid thresholds	200, updated rules saved
PUT /api/alerts/rules	Threshold below 0	400, validation error
GET /api/export/csv	7 day range	200, CSV Content-Type
GET /api/export/csv	100 day range	202, job ID returned

Neo4j / Data Layer Tests

Test	Cypher Used	Expected
Save reading to Neo4j	CREATE (r:Reading {...})	Node created with all fields
Query latest reading	MATCH ... ORDER BY ts DESC LIMIT 1	Most recent reading returned
Alert created on breach	MATCH reading → CREATE alert	Alert node linked to reading
Cross-user data isolation	User A queries User B's device	Empty result (no relationship)
Reading count after 100 inserts	MATCH (d)-[:HAS_READING]->(r) RETURN count(r)	100
Delete device cascade	DETACH DELETE device	All readings + alerts also removed

Socket.io Real-time Tests

Test	Method	Expected
Client subscribes to device	emit('subscribe', {deviceId})	Joins room, receives future updates
ESP32 publishes, client receives	MQTT publish → backend	Client receives sensor:update within 500ms
Alert fires, client notified	Threshold breach	Client receives alert:new event
Client disconnects, reconnects	Socket drop + reconnect	Re-subscribes automatically, data resumes
100 concurrent socket clients	Load test	All clients receive same update, no drops

UI / E2E Tests (Playwright or Cypress)

Flow	Steps	Expected
Login → see dashboard	Login → /	Dashboard loads, device data visible
Gas card click	Click PM2.5 card	Navigates to /gas/pm25
Time range change	Click "24H" button	Chart data updates to 24h window
Alert acknowledge	Click Acknowledge	Alert disappears from active list

Flow	Steps	Expected
CSV export	History → Export → CSV → Download	File downloads with correct columns
Theme toggle	Settings → Dark/Light	Theme changes, persists on refresh
Device rename	Devices → Edit → Save	New name appears everywhere
Password change	Settings → Change Password → Submit	Success message, old password rejected

14. Alert Engine Logic

MQTT message received → Node.js backend

Save reading to Neo4j

For each pollutant in reading:

```

    Is value > criticalThreshold? → Level = CRITICAL
    Is value > warningThreshold? → Level = WARNING
    Otherwise                      → No alert

```

If alert level found:

```

    Check Redis for cooldown key: alert:{deviceId}:{pollutant}
    If key exists (cooldown active) → skip
    If key not exists:
        Create Alert node in Neo4j
        Set Redis key with TTL = cooldownMinutes * 60
        Socket.io emit('alert:new') to device room
        Send email if notification enabled

```

If value returned to safe after alert:

```

    Find unresolved alert for this pollutant
    Mark as resolved in Neo4j
    Socket.io emit('alert:resolved')

```

15. Deployment Steps (Ubuntu, No Docker)

1. Server setup

```

sudo apt update && sudo apt upgrade -y
sudo apt install nginx certbot python3-certbot-nginx nodejs npm git -y

```

```

# 2. Install Neo4j (as above in Section 10)

# 3. Install Mosquitto MQTT broker
sudo apt install mosquitto mosquitto-clients -y
sudo systemctl enable mosquitto
# Config: /etc/mosquitto/mosquitto.conf

# 4. Clone and install project
git clone https://github.com/your-org/aewis-web.git /var/www/aewis
cd /var/www/aewis
npm install

# 5. Build frontend
cd frontend && npm install && npm run build
# Outputs to frontend/dist/ → served by Nginx

# 6. Set environment variables
cp backend/.env.example backend/.env
nano backend/.env
# NEO4J_PASSWORD=yourpassword
# MQTT_BROKER=mqtt://localhost:1883
# JWT_SECRET=your-long-random-secret
# EMAIL_USER=your@gmail.com
# EMAIL_PASS=your-app-password

# 7. Start backend with PM2
npm install -g pm2
pm2 start ecosystem.config.js
pm2 startup && pm2 save

# 8. Configure Nginx + SSL
sudo cp nginx.conf /etc/nginx/sites-available/aewis
sudo ln -s /etc/nginx/sites-available/aewis /etc/nginx/sites-enabled/
sudo certbot --nginx -d app.aewis.in
sudo nginx -t && sudo systemctl reload nginx

```

16. Environment Variables

```

# Neo4j
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=your_neo4j_password

# MQTT

```



```
MQTT_BROKER=mqtt://localhost:1883
MQTT_USERNAME=aewis
MQTT_PASSWORD=your_mqtt_password

# Auth
JWT_SECRET=your_very_long_random_secret_here
JWT_EXPIRY=15m
REFRESH_TOKEN_EXPIRY=30d

# Email
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=alerts@aewis.in
EMAIL_PASS=gmail_app_password

# Server
PORT=3000
NODE_ENV=production
FRONTEND_URL=https://app.aewis.in
```

End of Web Dashboard PRD v3.0